

InvoiceFlow

Enterprise Invoice Payment and Reconciliation

SC6113 Individual Assignment

Student name: _____ Student ID: _____

Introduction

InvoiceFlow is a financial decentralized application that connects an enterprise invoice to a customer-specific ETH payment and an independently reconciled receipt. A Solidity contract enforces settlement rules, MetaMask-compatible wallet access authorizes transactions, and a Flask backend verifies blockchain evidence. The implemented workflow covers issuance, payment, cancellation, merchant withdrawal, transaction history and reconciliation export. The prototype demonstrates programmable payment controls without assuming that an attractive interface or a transaction hash proves settlement.

Problem Statement

A small consultancy may issue several invoices and receive payments from different wallet addresses. A standalone transfer does not establish which invoice it belongs to, whether it covers the exact amount, or whether a reported payment succeeded. Manual matching creates opportunities for duplicate entries and incorrect receivable balances. A centralized paid/unpaid flag is also easy to update without independently checking its evidence. The target users are merchants reviewing receivables and customers paying assigned invoices.

Objectives

The project aims to: (1) bind a unique merchant reference to an intended payer, amount and deadline; (2) execute exact-value payments and prevent unauthorized or repeated payment; (3) separate merchant revenue withdrawal from invoice settlement; (4) reconcile from blockchain events and state rather than browser claims; and (5) detect changes to invoice metadata while keeping descriptions outside contract storage.

Project Scope

The selected scope is a single-chain, ETH-denominated invoice prototype. It does not provide fiat conversion, tax-invoice certification, refunds or partial payment. Automated evidence was collected on a local Ganache EVM. The application includes a real MetaMask connection path and Sepolia deployment configuration; a live extension and public-testnet run have not been verified. This distinction is retained throughout the evaluation.

System Architecture

InvoiceFlow separates the payment path from the information and reconciliation paths. The browser loads HTML/CSS/JavaScript from Flask. Wallet-authorized writes travel through ethers.js to the configured Ethereum contract. Flask separately uses Web3.py to read contract logs and state, stores descriptions and indexing checkpoints in SQLite, and supplies verified records to the interface. No user private key is sent to Flask. The contract, rather than a database row or hidden button, decides who may pay, cancel or withdraw.

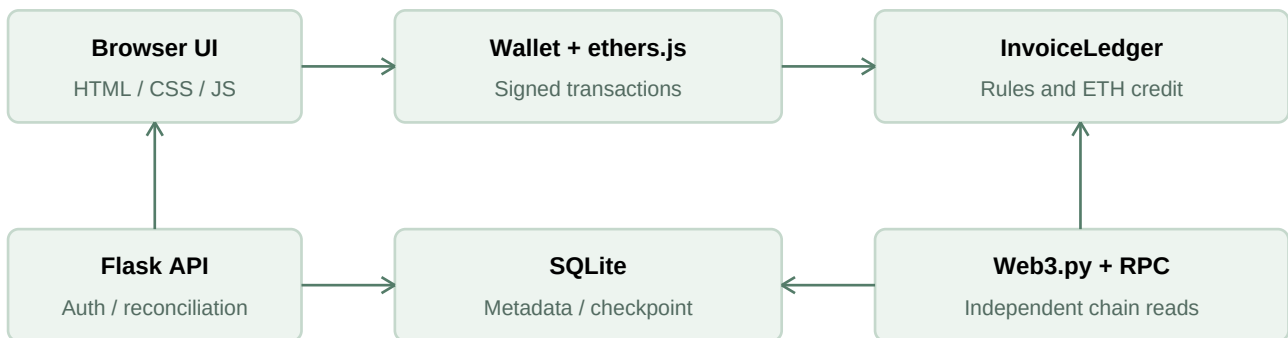


Figure 1. Separate wallet-authorized payment and server-verification paths. Descriptions are auxiliary off-chain data.

Technologies Used

Solidity 0.8.26 implements the business rules. Compilation uses an optimizer with 200 runs and a Shanghai EVM target. ethers.js 6.13.5 binds the ABI, contract address and wallet signer. Flask 3.1.0 handles routes, authentication and API responses; Web3.py 7.8.0 provides server-side RPC reads. SQLite stores metadata, event records and checkpoints. Ganache 7.9.2 supplies a deterministic local EVM. Python/pytest and a headless Chrome browser provide integration evidence.

Component	Authoritative responsibility
Smart contract	Invoice status, exact amount, caller permissions, merchant credit
Wallet / ethers.js	Account authorization and transaction/message signing
Flask / Web3.py	Independent reads, issuer authentication, reconciliation and CSV
SQLite	Auxiliary descriptions, replayable event cache and checkpoint
Browser UI	User input, progress, errors and rendering verified records

Trust Boundaries

The server RPC URL is not exposed in public browser configuration. The selected network and address must match the deployment. The local test-wallet adapter is enabled only for chain 31337 on localhost and is visibly labelled. Public Sepolia configuration disables it. Wallet addresses and transactions remain publicly observable even when descriptions are stored off-chain.

Smart Contract Design

InvoiceLedger stores an Invoice struct containing merchant, payer, amount, dueAt, status and documentHash. The invoice ID is keccak256 of ABI-encoded merchant address and the keccak256 reference. This namespaces references by merchant and makes reuse detectable without storing the reference text. State changes emit events with indexed invoice and wallet fields so a backend can retrieve an audit trail without iterating a contract-wide array.

Operation	Conditions	Result
Create	Positive amount; future deadline; distinct non-zero payer; non-empty hashes; unused ID	Open invoice; immutable core fields
Pay	Open; designated payer; not expired; exact msg.value	Paid; merchant credit and liability increase
Cancel	Open; issuing merchant	Cancelled; further payment prohibited
Withdraw	Positive caller credit; reentrancy lock	Credit and liability reduced; Ether transferred
Pause	Administrator only	Creation/payment blocked; exits remain available

Funds and Accounting Invariant

Payments are held as merchant credits rather than immediately forwarded. totalLiability equals the sum of unpaid-withdrawal credits after successful operations; the ETH balance must cover that liability. An externally forced ETH transfer could make balance greater than liability, so equality is not assumed for every possible EVM state. The tested ordinary payment/withdrawal sequences maintain equality. No administrator sweep function can divert merchant credit.

Withdrawal Safety

withdraw checks the caller credit, clears it, reduces liability, and then invokes the receiving address. A nonReentrant lock supplements this checks-effects-interactions order [1]. A failed transfer reverts the transaction and restores its state. Rejecting and reentrant contract receivers were tested. Pausing cannot trap already-earned merchant funds because withdrawal remains enabled.

Deadline and Status Semantics

The stored enum is Missing, Open, Paid or Cancelled. Overdue is a derived display state when blockchain time exceeds dueAt; no scheduled transaction is required. The contract rejects late payment. Confirming and Settled describe the backend confirmation policy, while the contract state is already Paid. A confirmation threshold reduces premature reporting but is not a guarantee against every reorganization.

Application Design

The dashboard groups outstanding receivables, reconciled payments, current-wallet credit and ledger checkpoint. The invoice table supports reference/address search, status filtering and issuer/payer views. A details dialog exposes wallets, the content hash and transaction identifiers. Customer actions and issuer cancellation are shown according to wallet role; these interface conveniences are backed by independent contract checks. Responsive CSS supports a narrow mobile viewport without page-level overflow.

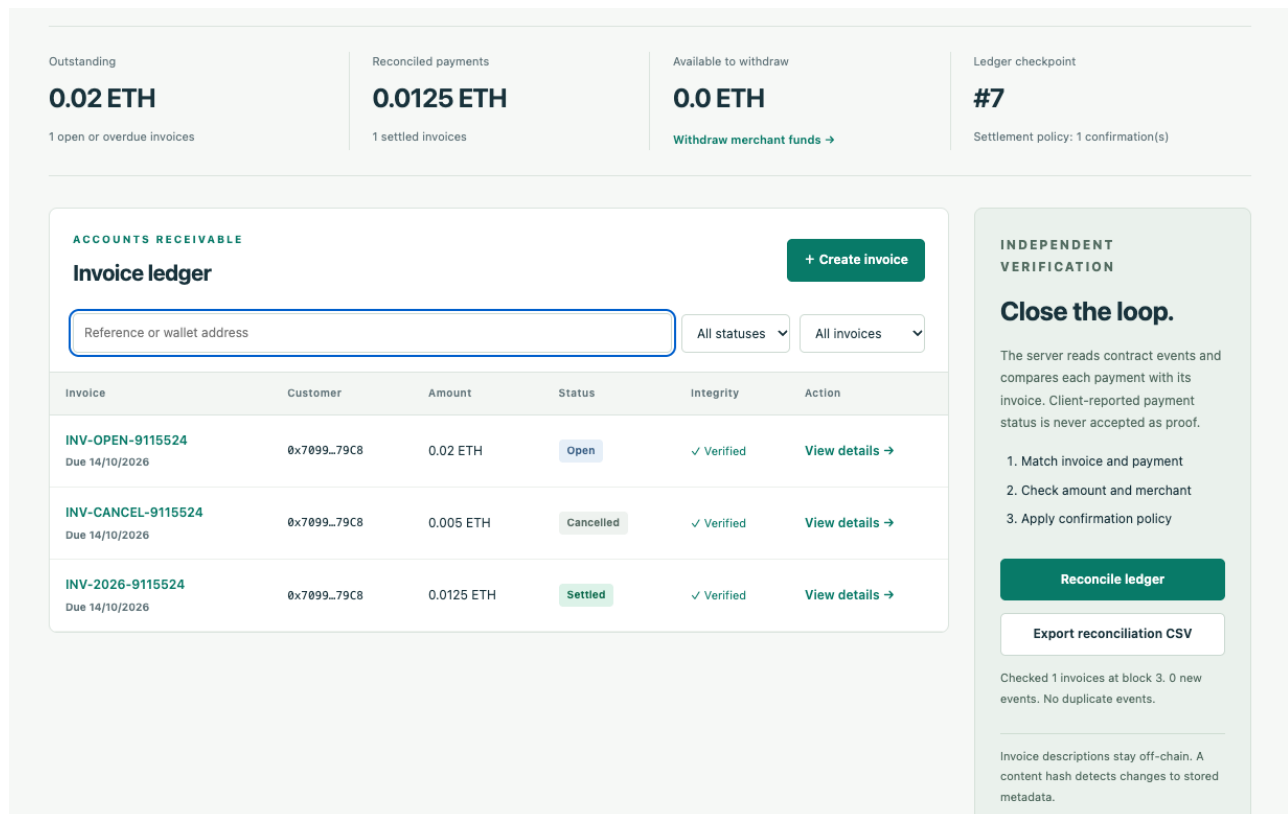


Figure 2. Observed local dashboard after issue, payment, withdrawal and cancellation. All rows are backed by local-chain transactions.

Implementation

The creation flow validates input, obtains a nonce-bound issuer signature, hashes normalized metadata, sends createInvoice and waits for its receipt. Only after chain success does the backend accept metadata. The payment flow sends the exact integer wei value and waits for a successful receipt before refreshing. The transaction panel distinguishes wallet approval, submission, confirmation and failure. Account/network changes clear the active signer and require reconnection [2].

Error Handling and Recovery

Declined wallet requests, invalid amounts, unavailable RPC services, wrong networks and contract reverts produce explanatory messages. Zero-value form input is rejected before submission, while the contract independently rejects it. If creation succeeds but metadata upload fails, a browser recovery copy permits issuer-authenticated reattachment; the UI does not pretend the committed transaction was rolled back. Missing or corrupted descriptions are labelled unverified without preventing chain-based accounting.

Backend Integration and Reconciliation

The backend checks RPC chain ID and deployed code, then scans only the configured contract from its deployment block. Each event is uniquely keyed by network/contract scope, transaction hash and log index. Reads are pinned to a captured head block. Payment events are matched to invoice amount, merchant and Paid state; the required confirmation count determines whether the record is Settled. The browser cannot create a payment proof by submitting a paid flag.

Idempotency and Rollback Recovery

A saved checkpoint records block number and hash. A second scan resumes after it, and uniqueness constraints protect against duplicate records. If the checkpoint is no longer canonical or is beyond the chain head, indexed events are cleared and replayed. The backend checks the head hash before committing and after invoice reads, so a detected change asks for retry. The integration test rolls back a real local payment and confirms that its cached receipt disappears and the invoice returns to Open.

Metadata Integrity and Authentication

Canonical JSON uses a fixed field order: reference, description, lowercase payer, integer amountWei string and integer dueAt string. The contract commits SHA-256 of its UTF-8 bytes. Flask recomputes the digest, verifies the issuer and chain fields, and checks integrity again on read. A single-use five-minute challenge includes origin and chain ID; signature recovery identifies the wallet. Metadata writes require a session CSRF token. This limited personal-sign design is not a complete SIWE implementation.

Security and Ethical Considerations

The implementation validates input at browser, server and contract boundaries. SQL uses parameters, browser values are escaped, and CSV references resembling formulas are prefixed. Session cookies are HttpOnly and SameSite Strict; production configuration enables Secure cookies, an explicit trusted host and a separate secret [3]. A self-only script policy avoids remote script loading. Production rate limiting, stronger access controls for private business data and a professional contract audit remain necessary.

Privacy Boundaries

Off-chain storage limits public-chain text but does not make descriptions confidential: the prototype exposes them through read APIs and uses fictitious examples. Hashing predictable content can permit guessing. Public wallets, amounts and timing may reveal relationships. A production system would require authenticated read access, retention controls and carefully designed confidential data storage.

Testing and Results

The recorded local run passed 61 automated checks: 29 contract tests, 19 backend integration tests and 13 browser checks. Contract tests use an isolated Ganache instance; backend tests deploy independent fixture contracts against localhost. Browser tests execute real transactions through the explicitly labelled development provider. The injected wallet-rejection fixture tests the client error path, not the real MetaMask extension. Raw JSON, JUnit XML, logs and screenshots accompany the source.

Layer	Selected checks	Observed result
Contract	Exact/under/overpayment; duplicate; authorization; cancellation; expiry; pause	29 / 29 passed
Contract security	Reentrant receiver; rejected transfer rollback; 12-payment liability invariant	Included in contract suite
Backend	Nonce replay/expiry; CSRF; issuer; metadata tamper; confirmations; rollback; CSV	19 / 19 passed
Browser	Issue, pay, withdraw, cancel, filter, export, mobile overflow, rejected request	13 / 13 passed
Public network	Actual MetaMask extension and Sepolia deployment	Not executed

Gas and Local Response Measurements

The compiled deployment used 714,039 gas. Invoice creation used 140,057, exact payment 82,546, and withdrawal 36,031. These are observed transaction costs under the stated compiler and local EVM, not quoted fees. The operations contain no user-count-dependent loops. Actual fee depends on network gas price and execution context.

Measure	Observed value	Conditions
GET /api/invoices median	27.56 ms	20 sequential warm requests
Sample 95th percentile	51.32 ms	Nearest-rank order statistic
Dataset	3 invoices	Local Flask server and Ganache
Public-network latency	Not measured	No throughput or Sepolia inference

Interpretation

Tests support the implemented rule boundaries and selected attack scenarios, rather than proving universal security. The small latency sample demonstrates local responsiveness only; it is not a load test, statistical confidence interval or production benchmark. Browser screenshots show an actual created invoice, settled payment, mobile layout and input failure. Repeated scans did not duplicate events; tampering removed the integrity indicator.

Challenges Encountered

The implementation had to keep two authoritative domains distinct: the chain controls funds and statuses, while SQLite provides recoverable descriptions and indexing. A successful write followed by a failed metadata upload must not be reported as an entirely failed transaction. The resulting recovery mechanism preserves the distinction. Authentication also had to prove issuer control rather than trust a typed wallet address.

Integration and Validation Challenges

ethers.js version 6 differs from the version 5 patterns in parts of the course material; a single API family is used consistently. The compiler ABI and deployment address are generated from the current contract. Tests identified a malformed Flask configuration response during development; it was corrected and the full backend suite passed. Reconciliation required replay-safe keys and checkpoint-hash verification so a locally observed payment could be removed after rollback. All demonstrations distinguish local-chain proof from public-network deployment.

Limitations

ETH volatility makes fixed ETH invoices different from fiat-denominated receivables. The prototype omits refunds, partial payments, amendments, tax and currency conversion. A merchant must initiate withdrawal; a rejecting contract receiver needs to correct its receiving behavior. The administrator can suspend new business, which is a centralized control even though merchant funds cannot be redirected. Public chain data still reveals wallet activity.

Operational Limits

The indexer uses an in-process lock and SQLite; it is intended for a small single-instance deployment. Scanning is incremental, but every refresh still reads all indexed invoices. Multi-worker coordination, pagination, RPC rate limits, durable hosted storage, service monitoring and production abuse protection require further engineering. A fixed confirmation count is an application policy. The real MetaMask extension, Sepolia transaction lifecycle, cloud hosting and usability with external users remain unverified.

Future Improvements

The next step is a documented Sepolia run with dedicated test wallets, wallet screenshots and explorer hashes. A subsequent version could settle in a reputable test ERC-20 token, with allowance and decimal handling tested separately. Larger deployments need a durable database, a background indexer, bounded/paginated reads and reorganization monitoring. Business extensions should define amendment, refund and dispute rules before implementation. Formal security review and confidential enterprise-data access should precede any real-value use.

Conclusion

InvoiceFlow implements an end-to-end financial DApp that binds invoices to authorized, exact-value payments and makes their reconciliation independently observable. Its main contribution is the combination of on-chain settlement rules, issuer-authenticated metadata commitments, replay-safe indexing and explicit confirmation handling. The implementation uses the course principles of Solidity structures, permissions, events, wallet integration, Flask/SQLite and security testing. The 61 passing local checks provide reproducible evidence for the selected behaviors while leaving public deployment and production assurance as separate tasks.

Rubric Evidence Summary

The project connects a concrete receivables problem to measurable rules and a working browser flow. The strongest evidence for assessment is the contract and its adversarial tests, the independently reading backend, the actual transaction screenshots, and the documented limitations. The mapping below identifies evidence, not a predicted mark.

Criterion	Marks	Evidence
Problem and objectives	10	Pages 1–2; SME payment-matching scenario
Smart contract design	20	Page 3; source, receiver attack tests, liability invariant
DApp functionality	25	Issue/pay/cancel/withdraw/reconcile/export; browser evidence
Frontend interface	10	Page 4; desktop/mobile screenshots and validation
Backend integration	10	Page 5; authentication, SQLite, chain reads and recovery
Testing and evaluation	10	Page 6; raw results, gas and bounded latency sample
Report quality	10	All required sections, figures, references and limitations
Innovation	5	Metadata commitment + rollback-aware independent reconciliation

References

- [1] Solidity documentation, version 0.8.26. Security Considerations. <https://docs.soliditylang.org/en/v0.8.26/security-considerations.html> (accessed 7 October 2026).
- [2] ethers.js documentation, version 6. Getting Started. <https://docs.ethers.org/v6/getting-started/> (accessed 7 October 2026).
- [3] Flask documentation, version 3.1. Security Considerations. <https://flask.palletsprojects.com/en/stable/web-security/> (accessed 7 October 2026).
- [4] SC6113 course materials: W1 Blockchain Basic; W2 DAPP; W3 to 4 Solidity and DAPP Programming; W5 Full Stack Programming; W6 Cyber Security Smart Contract; Web3 Blockchain DApp; Document.pdf. Supplied teaching materials.
- [5] SC6113 Individual Assignment: Financial Decentralized Application. Supplied assignment brief and 100-mark rubric.